

Sentiment analysis

Abdessamed Qchohi
EURECOM

Abdessamed.Qchohi@eurecom.fr

Francesco Giannuzzo
EURECOM

Francesco.Giannuzzo@eurecom.fr

Alessio Giuffrida
EURECOM

Alessio.Giuffrida@eurecom.fr

Matteo Ghia
EURECOM

Matteo.Ghia@eurecom.fr

Abstract—This project addresses the task of sentiment analysis on a dataset of tweets. The goal is to classify tweets with positive, neutral, or negative labels, reflecting the sentiment of the user writing the tweet. We experimented with various models, like traditional machine learning models such as SVM and logistic regression, a LSTM model, which is a type of Recurrent Neural Network (RNN), and RoBERTa, a transformed-based model. We also tackled the additional task of identifying the words inside the tweet which are most responsible for the predicted sentiment.

I. INTRODUCTION

Sentiment analysis is a Natural Language Processing (NLP) technique used to determine the emotional tone or opinion expressed in a piece of text. In the recent years, sentiment analysis has been applied in the context of social networks, where there are a large amount of textual data, which can be exploited to evaluate the general public’s opinion about certain topics. In this work we apply sentiment analysis to tweets from Figure Eight’s Data for Everyone platform [1]. Each tweet can be classified according to one of the following classes: *positive*, *neutral* and *negative*. In addition to this task, this project will also aim at detecting the words of a tweet that have the largest influence on the attributed sentiment.

II. DATASET ANALYSIS

The dataset is composed of a training set and an unlabeled test set. The training set is made up of 24732 rows, each containing a unique tweet ID, the text of the tweet, the selected text (for the additional task) and the label. Our work is focused mainly on the training set. The first step was analysing the distribution of the labels in the training set: Fig. 1 shows that the most frequent class is *neutral*, whereas the other two classes (*positive* and *negative*) have similar frequencies. We can therefore conclude that the dataset is slightly unbalanced, but this will not be a problem as the chosen evaluation metric, the macro F1 score, treats each class with equal significance. To check what are the most frequent words for each label, we plotted three separate WordClouds, as shown in Fig. 2. This was done after doing simple preprocessing steps, like converting the text to lowercase, removing punctuations and stopwords (with the exception of *not*).

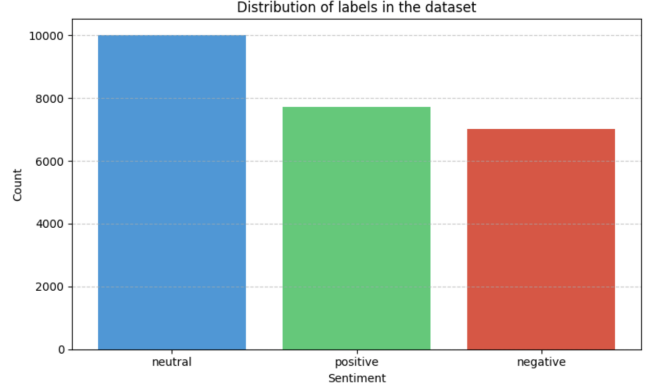


Fig. 1. Labels’ distribution

III. DATA PREPROCESSING AND PREPARATION

For each tweet the following preprocessing steps were performed:

- removal of links, hashtags, mentions;
- conversion of emojis in textual form;
- conversion of all the words to lowercase;
- extension of the contractions (for example *can’t* becomes *cannot*);
- removal of punctuation, numbers and redundant whitespaces;
- fixed non-spaced words (for example *reallymuch* becomes *really much*);
- fixed words with an excessive number of consecutive characters (for example *haaaaaateee* becomes *hate*);
- fixed misspelled words;
- performed stopwords removal (with the exception *not* since it heavily influences the predicted label);
- applied both stemming and lemmatization: stemming is the process of reducing a word to its root or base form by removing suffixes or prefixes, whereas lemmatization is the process of reducing a word to its dictionary base form using vocabulary and morphological analysis. We then checked 100 words randomly to verify the results. We noticed that stemming caused some spelling errors when truncating the words, so in the end we chose lemmatization;

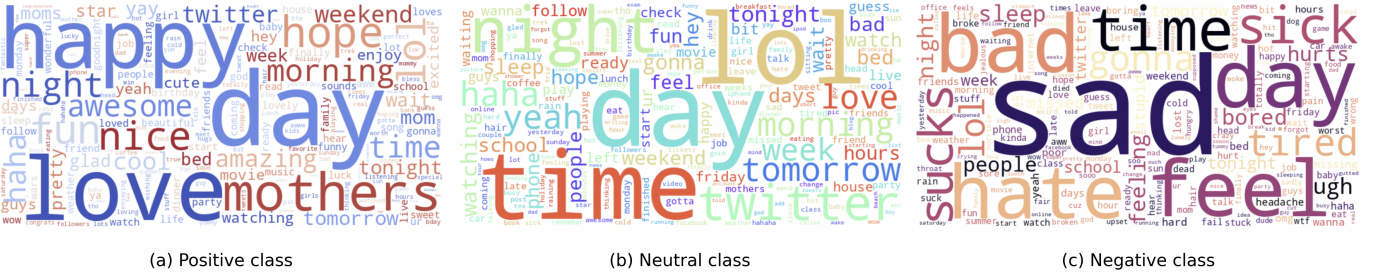


Fig. 2. WordClouds for the three different classes

- performed tokenization to convert each tweet into a list of tokens.

We split the dataset into a training, a validation and a test set using a 70/15/15 split for all models.

IV. MODEL SELECTION

To address the task we tested several models, ranging from traditional machine learning models to Recurrent Neural Networks (RNN) and transformer-based architectures:

- SVM: a supervised machine learning algorithm used for classification tasks. The SVM finds the optimal decision boundaries that best separate the different sentiment classes in the feature space. Since standard SVMs are binary classifiers, multi-class classification (such as 3-way sentiment analysis) is typically handled using strategies like one-vs-one or one-vs-rest.
- Logistic regression: a supervised machine learning algorithm commonly used for classification tasks. For multi-class classification (like with three labels), the algorithm typically uses a softmax function in a variation called multinomial logistic regression.
- LSTM [2]: Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) designed to learn and remember patterns in sequential data, such as text, time series, or speech. Unlike standard RNNs, LSTMs are capable of learning long-range dependencies by using a specialized architecture that includes memory cells and gating mechanisms (input gate, forget gate, and output gate). These gates control the flow of information, allowing the LSTM to retain important information over time and forget irrelevant data, which helps prevent issues like the vanishing gradient problem that often affects traditional RNNs.
- RoBERTa [3]: a transformer-based language model, which is an extension of BERT (Bidirectional Encoder Representation from Transformers), that is designed to understand the context of words in a sentence by looking at both the left and right sides simultaneously, a technique known as bidirectional encoding. BERT is pre-trained on large text corpora using masked language modeling and next sentence prediction, and can be fine-tuned for various NLP tasks like sentiment analysis, text classification, and question answering. RoBERTa builds on BERT

by removing some of its limitations and enhancing its training strategy.

In order to evaluate the performance of the models we used the macro F1 score, which is a metric used especially in multi-class settings: it is defined as the unweighted mean of the F1 scores computed independently for each class, treating them equally, regardless of how many samples belong to each one.

A. SVM

Before passing the data to the model, for each tweet we performed TF-IDF vectorization on the list of tokens: this procedure computes for each term a score that reflects the importance of that term in a document. When performing hyperparameters tuning on the model, we also considered the hyperparameters of the TF-IDF function. The multiclass classification with SVM is handled according to a one-vs-one scheme, meaning that it breaks the multi-class problem into multiple binary classification problems. Specifically, for a task with K classes, the one-vs-one approach trains a separate SVM for each pair of classes.

We performed hyperparameter tuning with grid search on the hyperparameters in Table I, obtaining the best configuration of hyperparameters: **tfidf_max_features: 5000**, **tfidf_ngram_range: (1,1)**, **svm_C: 1.0**, **svm_kernel: 'rbf'**, **svm_gamma: 'scale'**, and metric's value on the validation set: **macro F1 score: 0.703**.

TABLE I
HYPERPARAMETERS SVM

Model	Hyperparameter	Values
SVM	tfidf_max_features	{5000, 10000}
	tfidf_ngram_range	{(1,1), (1,2)}
	svm_C	{0.1, 1.0}
	svm_kernel	{'rbf', 'poly'}
	svm_gamma	{'scale', 'auto'}

We retrained with the best hyperparameters on the entire training set (including the validation set) and we obtained the following results on the test set: **macro F1 score: 0.694**. Fig. 3 shows the confusion matrix for the labels predicted on the test set. We can see that the model struggles to classify tweets with the negative label, but overall the performance is acceptable. However, it is possible to obtain better result using other models.

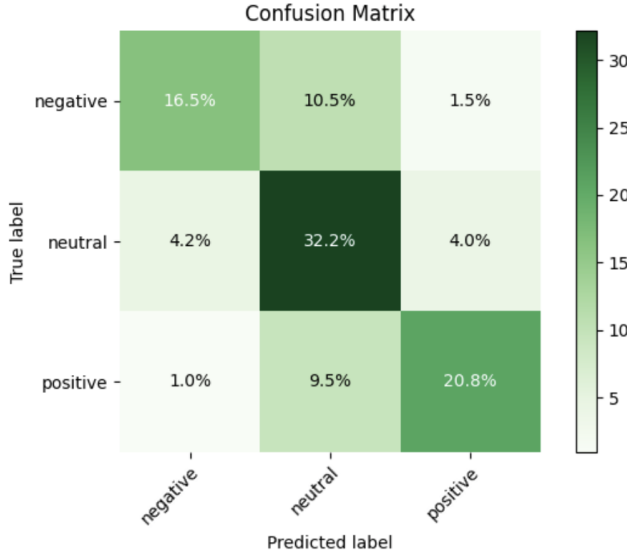


Fig. 3. Confusion matrix for SVM

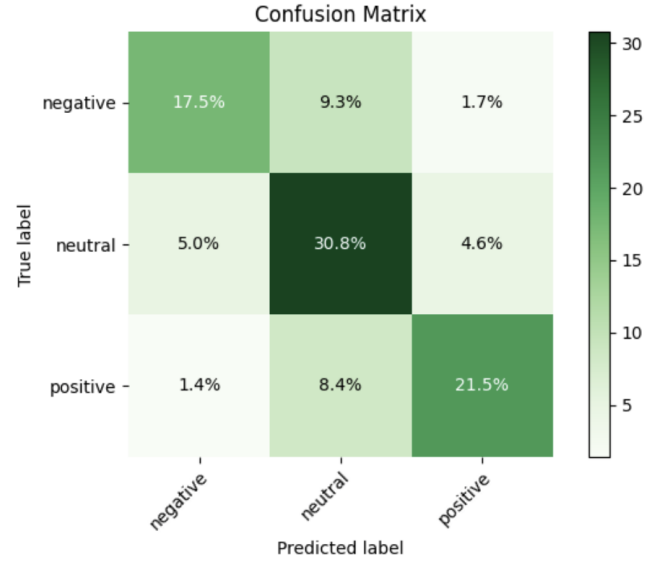


Fig. 4. Confusion matrix for Logistic Regression

B. Logistic Regression

The logistic regression model we used automatically handles multiclass classification by implementing what is called multinomial logistic regression. This version of logistic regression directly models the probability of each class using a softmax function, which outputs a probability distribution over all classes. A score is calculated for each class and is then passed through the softmax function to produce probabilities for each class, choosing the one with the highest predicted probability. As we did for SVM, before passing the data to the model, we performed TF-IDF vectorization on the lists of tokens.

We performed hyperparameter tuning with grid search on the hyperparameters in Table II, obtaining the best configuration of hyperparameters: **'tfidf_max_features': 10000**, **'tfidf_ngram_range': (1, 2)**, **'lr_C': 1.0**, **'lr_solver': 'liblinear'**, and metric's value on the validation set: **macro F1 score: 0.724**.

TABLE II
HYPERPARAMETERS LOGISTIC REGRESSION

Model	Hyperparameter	Values
Logistic regression	tfidf_max_features	{5000, 10000, 20000}
	tfidf_ngram_range	{{(1,1), (1,2)}}
	lr_C	{0.01, 0.1, 1.0, 10.0}
	lr_solver	{'liblinear', 'lbfgs'}

We retrained with the best hyperparameters on the entire training set (including the validation set) and we obtained the following results on the test set: **macro F1 score: 0.706**. Fig. 4 shows the confusion matrix for the labels predicted on the test set. The results are slightly better than SVM, even though not by much.

C. LSTM

We built a vocabulary based on the terms appearing in the training and validation sets, so some terms in the test set will never be seen by the model, and we substituted them with an *UNK* token. We also used this token to pad all sequences of tokens up to the maximum sequence length in the training set. The model takes as input a size 32 batch of padded token ID sequences, which is converted into a dense vector using a learned embedding matrix: these embeddings capture semantic similarities between words. The embedding vectors are learned during training, just like the weights of other layers. The sequence of embeddings is passed through the LSTM layers: the LSTM processes the input one token at a time, maintaining a hidden state that remembers information from earlier tokens. Unlike standard RNNs, LSTMs use gates (input, forget, output) to preserve long-term information and avoid vanishing and exploding gradients. The final layer is a fully connected layer that outputs a vector of logits, representing the likelihood of each sentiment class. A softmax is then applied to interpret these scores as probabilities.

We performed hyperparameter tuning with grid search on the hyperparameters in Table III, obtaining the best configuration of hyperparameters: **embedding_dim: 16**, **num_layers: 2**, **step_size: 3**, **learning_rate: 1e-2**, and metric's value on the validation set: **macro F1 score: 0.687**. We used Adam as optimizer and StepLR as scheduler, and we trained for hyperparameters tuning for 5 epochs.

TABLE III
HYPERPARAMETERS LSTM

Model	Hyperparameter	Values
LSTM	embedding_dim	{16, 64, 128, 256}
	num_layers	{1, 2, 3, 5}
	step_size	{3, 5, 7, 10}
	learning_rate	{1e-2, 1e-3, 1e-4}

We retrained with the best hyperparameters on the entire training set (including the validation set) for 15 epochs and we obtained the following results on the test set: **macro F1 score: 0.702**. Fig. 5 shows the confusion matrix for the labels predicted on the test set. This model performs similarly to the SVM and logistic regression: this may be due to the fact that we did not have enough training data, and so the model does not learn effectively.

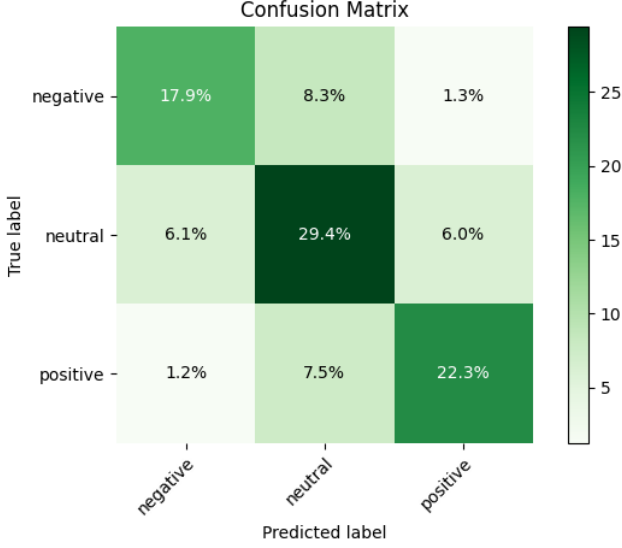


Fig. 5. Confusion matrix for LSTM

D. RoBERTa

The RoBERTa model was pretrained on raw texts in a self-supervised fashion: for this reason we passed the training data to the model without performing any kind of preprocessing. RoBERTa uses a tokenizer based on Byte-Pair Encoding (BPE), designed to efficiently handle large vocabularies, rare words, and out-of-vocabulary terms. The text is first split into UTF-8 bytes, which means every character can be represented: this makes RoBERTa robust to any input. Initially, each character or byte is treated as a separate token. The tokenizer has a precomputed list of frequent symbol pairs (learned during tokenizer training), and it iteratively merges the most frequent adjacent pairs into single tokens. The result is a fixed-size vocabulary, and all the inputs to the language model are broken down into the subword tokens from the vocabulary. We used RoBERTa base model, performing fine-tuning of all the layers. We performed hyperparameter tuning with grid search on the hyperparameters in Table IV, obtaining the best configuration of hyperparameters: **gamma: 0.1, step_size: 5, learning_rate: 1e-5, weight_decay 0.01**, and metric's value on the validation set: **macro F1 score: 0.812**. We used Adam as optimizer and StepLR as scheduler, with batch size 32 and we trained for hyperparameters tuning for 5 epochs.

TABLE IV
HYPERPARAMETERS ROBERTA

Model	Hyperparameter	Values
RoBERTa	gamma	{0.01, 0.1, 0.5}
	step_size	{3, 5, 7, 10}
	learning_rate	{1e-3, 1e-4, 1e-5}
	weight_decay	{0.01, 0.1}

We retrained with the best hyperparameters on the entire training set (including the validation set) for 15 epochs and we obtained the following results on the test set: **macro F1 score: 0.801**. Fig. 6 shows the confusion matrix for the labels predicted on the test set. These are the best results obtained, which we expected since RoBERTa is a very powerful model specifically designed to address sentiment analysis tasks.

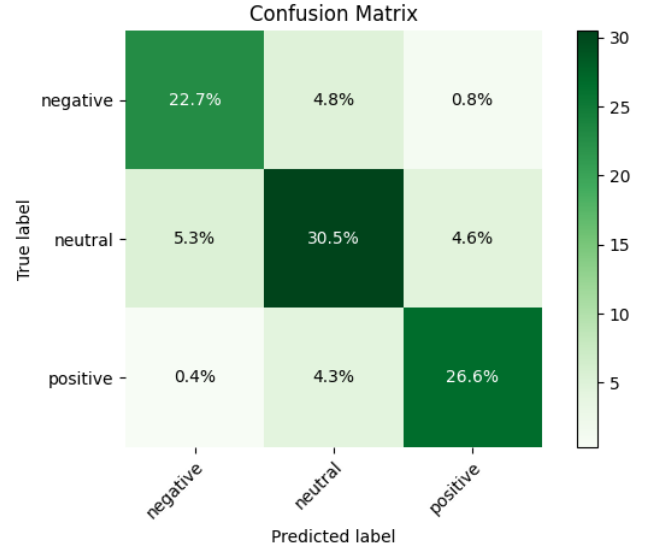


Fig. 6. Confusion matrix for RoBERTa

V. UNLABELED TEST SET

To assign labels to the unlabeled test set we used the *twitter-roberta-base-sentiment* pretrained model [4], which is a RoBERTa-based transformer model fine-tuned specifically for sentiment analysis on tweets. After assigning the labels, we evaluated the RoBERTa model we previously trained on this test set, obtaining the following result: **macro F1 score: 0.772**. This confirms the good generalization capabilities of RoBERTa.

VI. EXTRACTING RELEVANT TEXT

For the additional task, we used a tool that extracts and assigns a relevance score to the tokens in a tweet, called Transformers Interpret [5]. This tool takes as input the RoBERTa model we used previously and its tokenizer and then, when applied to a sample, returns a list of (word, importance score) pairs. The score reflects how much each word contributes to a sentiment prediction. This tool was applied to the test set obtained by splitting the original training dataset: we computed

a score for each word present in a tweet, we sorted the scores in decreasing order and we selected the top 5 words. Since there was a lot of variability in the number of words in the *selected text* feature, we chose 5 as a middle ground. Finally, we compared the relevant words we obtained in this way with the *selected text* column in the dataset, using as metric the Jaccard coefficient, which is defined as the intersection over union of the two sets. The result is a Jaccard coefficient of 0.35. This result is acceptable but can be improved, for example, by finding a good heuristic criterion to choose how many words to retain from the output of the Transformer Interpret.

VII. CONCLUSIONS AND FUTURE WORK

In this work we faced the sentiment analysis task on a dataset of tweets, in order to classify tweets as positive, neutral or negative. We experimented with traditional machine learning models, like SVM and logistic regression, and then with a LSTM model and RoBERTa, a transformed-based model. We obtained the best results with RoBERTa as we expected, since this is a very good model for this type of task. All the results are summarised in Table V.

TABLE V
RESULTS

Model	Validation macro F1 score	Test macro F1 score
SVM	0.703	0.694
Logistic Regression	0.724	0.706
LSTM	0.687	0.702
RoBERTa	0.812	0.801

We also employed the unlabeled test set by assigning labels using the *twitter-roberta-base-sentiment* pretrained model and evaluating our best model (RoBERTa) performance on it, obtaining satisfactory results. Finally, we tackled the additional task of finding the most significant words in a tweet that contribute to its sentiment using the Transformers Interpret tool: the metric considered was the Jaccard coefficient, for which we obtained a value of 0.35. As future work it would be useful to find a way to choose the optimal number of selected words to retain, maybe choosing a different number for each tweet, since the Jaccard coefficient is really sensible to mismatches.

REFERENCES

- [1] Figure Eight Inc., https://en.wikipedia.org/wiki/Figure_Eight_Inc..
- [2] Sepp Hochreiter, Jürgen Schmidhuber. "Long Short-Term Memory", Neural Computation, 1997.
- [3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach", 2019.
- [4] Twitter-roBERTa-base for Sentiment Analysis, <https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment>.
- [5] Charles Pierse, Transformers Interpret, <https://github.com/cdpierse/transformers-interpret>.